Stundendokumentation Dienstag, 03.11.2026: Rückgabe Klausur 1, Start mit der Klasse `Vertex`

Informatik LK, 1./2. Stunde | Einheit 2 „Objektorientierung und Spielprojekt“, erste Stunde nach den Herbstferien

Was wir gemacht haben

1. **Rückgabe Klausur 1:** Jede Person bekam ihren Satz mit dem passenden Extrablatt zurück. Klausur und Extrablatt still durchgesehen, Randzeichen und Erklärungen abgeglichen, die wichtigste Frage notiert und auf der Rückseite des Extrablatts den ersten Schritt angekreuzt.
2. **Drei Themen für alle:** Der Ausdruck `"2" + 3 + 4 * 5` Schritt für Schritt, dann `1 + 2 + "3" + 4 * 5` selbst gerechnet. Ein Fehlerbeleg nennt Eingabe, erwartetes und tatsächliches Ergebnis. In `main` ein Objekt erzeugen, eine Methode am Objekt aufrufen und den Rückgabewert ausgeben.
3. **Einstieg ins Spiel:** Ab heute bauen wir Kapitel für Kapitel nach dem Buch „Java will nur spielen“ von Sven Eric Panitz ein Spiel. Das BlueJ-Projekt `spiel` wächst in den nächsten Wochen.
4. **Aufgabe 1 bis 3 zu zweit in BlueJ:** Klasse `Vertex` mit Feldern und Konstruktor, ein erster Test in `main` mit vorhergesagten Ausgaben, dann `toString`. Ein Paar zeigte seine Lösung am Beamer.
5. **Schmiede:** die erste Lektion unter „Woran du als Nächstes arbeitest“ auf dem Extrablatt.
6. **Sicherung:** Merke-Tabelle vom Blatt gelesen, zwei Zeilen laut erklärt. Ausblick: Morgen lernt der Punkt rechnen und vergleichen.

Wenn du gefehlt hast

1. Hol dir dein Klausurblatt mit Extrablatt bei mir ab. Lies beides still und kreuz auf der Rückseite des Extrablatts deinen ersten Schritt an.
2. Rechne die beiden Ausdrücke `"2" + 3 + 4 * 5` und `1 + 2 + "3" + 4 * 5` selbst aus und prüfe sie im Code Pad (Lösung unten).
3. Hol dir das Blatt **„Vertex: Ein Punkt wird zum Objekt“** aus dem Gruppenordner KW45. Lege in BlueJ das Projekt `spiel` an und bearbeite **Aufgabe 1 bis 3**. In Aufgabe 2a schreibst du die Ausgaben **vor** dem Start auf.
4. Lies erst danach den Erwartungshorizont und übertrage den Merke-Kasten ins Heft.

Erwartungshorizont

Alle Ausgaben stammen aus Läufen mit `javac` und `java` 25.

Die drei Themen aus der Rückgabe

`1 + 2 + "3" + 4 * 5` ergibt `"3320"`: zuerst `4 * 5 = 20`, dann von links `1 + 2 = 3`, `3 + "3" = "33"` und `"33" + 20 = "3320"`.

Ein Beleg für einen Fehler nennt immer drei Dinge: die Eingabe, das erwartete und das tatsächliche Ergebnis. Beispiel für `schnittBestanden`: Bei `{2, 5, 9}` wird 7 erwartet, tatsächlich kommt 9. Bei `{1, 3}` wird 0 erwartet, tatsächlich bricht das Programm mit einer `ArithmeticException` ab.

Die Methode `main` zum Beispiel Bonuskarte gibt `true`, `true`, `false` und `3` aus.

Aufgabe 1: Felder und Konstruktor

1b) Eine Spielfigur bewegt sich in kleinen Schritten und wird mit Faktoren verrechnet. Die Werte sind deshalb meist nicht ganzzahlig, mit `int` gingen die Nachkommastellen verloren.

1a) und 1c)

```
public class Vertex {
    public double x;
    public double y;

    public Vertex(double xx, double yy) {
        x = xx;
        y = yy;
    }
}
```

Aufgabe 2: Ein erster Test

2a)

Ausgabe	v1.x	v2.y	v1.y	v1.y
erwartet und tatsächlich	`17.0`	`0.0`	`4.0`	`8.0`

Die Ausgabe heißt `17.0` und nicht `17`: Aus den Argumenten vom Typ `int` wurden Werte vom Typ `double`.

2b) Drei Objekte und drei Variablen (`v1`, `v2`, `vv`). Die Zeile `v1.y = 2 * v1.y;` ändert nur das Feld `y` des Objekts, auf das `v1` zeigt, von 4.0 auf 8.0. Sein `x` und die beiden anderen Objekte bleiben gleich.

2c) Etwas wie `Vertex@1dbd16a6`: Klassenname, `@` und ein Hashwert. Über die Koordinaten sagt das nichts, und der Wert ändert sich von Lauf zu Lauf.

Aufgabe 3: Der Punkt beschreibt sich selbst

```
public String toString() {
    return "(" + x + "," + y + ")";
}
```

3a) Jetzt steht dort `(17.0,8.0)`. `println` ruft für ein Objekt von selbst `toString` auf. Jede Klasse hat diese Methode von `Object`, unsere ersetzt sie. Das Wort **Überschreiben** fällt hier zum ersten Mal, die Erklärung folgt mit der Vererbung.

3b)

Ausdruck	Ergebnis
`(1 + 2) + "3"`	`"33"`
`1 + (2 + "3")`	`"123"`

Sobald ein String beteiligt ist, verkettet `+`, statt zu addieren. Die Klammer entscheidet, ob zuerst zwei Zahlen addiert werden (3, dann „33“) oder zuerst die 2 an den Text gehängt wird („23“, dann „123“). Das ist dieselbe Regel wie in `1 + 2 + "3" + 4 * 5`. Das Code Pad zeigt `"33"` mit Anführungszeichen, weil es so einen String kennzeichnet.

Zusatz: `public` oder `private`?

- **Dafür:** Kurz und schnell zu schreiben, `v.x` statt `v.getX()`. Ein Punkt hat keine Regel, die jemand verletzen könnte, jeder Wert ist erlaubt. Ein Spiel liest und ändert die Koordinaten tausendfach pro Sekunde.
- **Dagegen:** Jede Klasse kann die Werte ändern, auch versehentlich. Die innere Darstellung lässt sich später nicht mehr austauschen, ohne alle Stellen zu ändern, die `x` und `y` direkt benutzen.
- Das Buch bietet am Ende beides an, Felder und Zugriffsmethoden. Mehr dazu in der nächsten Stunde.

Die Klasse nach der Stunde

```java
public class Vertex {
    public double x;
    public double y;

    public Vertex(double xx, double yy) {
        x = xx;
        y = yy;
    }

    public String toString() {
        return "(" + x + "," + y + ")";
    }

    public static void main(String[] args) {
        Vertex v1 = new Vertex(17, 4);
        Vertex v2 = new Vertex(42, 0);
        Vertex vv = new Vertex(0, 0);
        System.out.println("v1.x = " + v1.x);
        System.out.println("v2.y = " + v2.y);
        System.out.println("v1.y = " + v1.y);
        v1.y = 2 * v1.y;
        System.out.println("v1.y = " + v1.y);
        System.out.println(v1);
    }
}
```

Wenn es hakt

- **Konstruktor mit `void` oder Rückgabetyp:** Dann ist es eine gewöhnliche Methode, und `new Vertex(17, 4)` scheitert. Der Konstruktor hat keinen Rückgabetyp.
- **`xx = x;` statt `x = xx;`:** Die Felder bleiben 0.0. Links steht, was gesetzt wird.
- **Parameter heißen `x` und `y`:** Dann überdecken sie die Felder, und `x = x;` setzt nichts. Mit `xx` und `yy` arbeiten, `this` kommt in der nächsten Stunde.
- **`main` ohne `static`:** BlueJ bietet `main` dann nicht im Kontextmenü der Klasse an.
- **`toString` mit großem T oder ohne `public`:** Ohne `public` meldet der Compiler schwächere Zugriffsrechte als bei `Object`. Mit falscher Schreibweise läuft alles, aber `println` zeigt weiter `Vertex@…`.

Merke

Begriff	Bedeutung
Feld	Eine Eigenschaft, die jedes Objekt der Klasse für sich hat
Konstruktor	Heißt wie die Klasse, hat keinen Rückgabetyp, setzt die Felder eines neuen Objekts
`new`	Erzeugt ein Objekt und ruft den Konstruktor auf
Punkt-Operator	`v1.x` fragt das Objekt in `v1` nach seinem Feld `x`
`toString`	Liefert die Textform eines Objekts, `println` ruft sie von selbst auf